WEEK 10 · WEB APPLICATIONS DEVELOPMENT

Back-End Development

Databases · Flat Files · SQL · NoSQL · Hybrid & Big Data

Cyber.SoHo Educational Hub · Schiller International University

Session Outline

Five lessons, one hour

1. Databases — History, Usage, Types, Providers
2. Flat Files — the honest ancestor
3. SQL — the fifty-year-old king
4. NoSQL — scale, schema, speed
5. Hybrid & Big Data — MapReduce and beyond

Target pace: ≈ 12 minutes per lesson

Learning Objectives

By the end of the hour you will be able to…

- ✓ Sketch the timeline of databases from tape to cloud
- ✓ Explain WHY applications need a back-end at all
- ✓ Read and write basic SQL statements
- ✓ Recognise the four families of NoSQL
- ✓ Describe Big Data and the MapReduce pattern

Where does the data live?

Your phone is a window, not a warehouse

- The photo you scrolled past is not on your phone
- The message you just received is not on your phone
- On the other side of the window sits a back-end
- A back-end = code that runs + a database that remembers

> *Today we talk about the remembering.*

LESSON 1

Databases

History · Usage · Types · Providers

What is a Database?

The core idea, before we open the textbook

> *A place that remembers. Nothing more mystical than that — what changed over 60 years is HOW it remembers, and HOW MUCH at the same time.*

History (1/2) · 1950s — 1970s

The foundations were laid

- 1950s — Magnetic tape, sequential access only
- 1960s — Hierarchical databases (IBM IMS for NASA Apollo)
- **1970 — Edgar F. Codd publishes the Relational Model**
- 1970s — First commercial relational engines appear

History (2/2) · 1980s — 2020s

Standardisation → open source → scale → AI

- 1986 — SQL standardised by ANSI and ISO
- 1980s–90s — RDBMS golden age: Oracle, Db2, SQL Server
- 1995–96 — MySQL and PostgreSQL open the floodgates
- 2000s — NoSQL wave: MongoDB, Cassandra, Redis
- 2010s — Cloud-native: DynamoDB, BigQuery, Snowflake
- 2020s — Vector / AI-native: Pinecone, Weaviate, pgvector

Why We Use Databases

The four reasons — memorise these

1. Persistence — data survives power loss
2. Concurrency — many users at once, no collisions
3. Integrity — rules enforced, garbage refused
4. Performance — indexes make one-in-a-billion instant

Major Providers (1/2) · Relational

The SQL engines you must know by name

Commercial heavyweights

- Oracle Database
- Microsoft SQL Server
- IBM Db2

Open source & embedded

- MySQL
- MariaDB
- PostgreSQL (recommended default)
- SQLite (mobile, embedded)

Major Providers (2/2) · NoSQL & Cloud

Specialised tools for specialised jobs

NoSQL engines

- Document — MongoDB, Couchbase, Firestore
- Key-Value — Redis, Memcached, DynamoDB
- Wide-column — Cassandra, Bigtable
- Graph — Neo4j, Neptune

Cloud & AI-native

- Managed — RDS, Aurora, Cosmos DB
- Warehouse — BigQuery, Snowflake, Redshift
- Vector — Pinecone, Weaviate, pgvector

Real-Life Example · Library Catalogue

Why this example works for beginners

- Every student understands lending a book
- Shows insert, update, query, aggregate on one dataset
- Four operations every DBMS offers, everywhere
- Python dict ↔ row · Python list ↔ table

KEY

A database is a place that remembers at scale.

The four reasons to use one are Persistence, Concurrency, Integrity, Performance. Default to PostgreSQL unless you have a specific reason not to.

LESSON 2

Flat Files

The honest ancestor of all databases

What is a Flat File?

Plain text, one record per line

- CSV — Comma-Separated Values
- TSV — Tab-Separated Values
- JSONL — one JSON object per line
- No engine, no server, no SQL — just bytes on disk
- The Esperanto of data — every language reads them

When a Flat File Fits (and When It Doesn't)

The honest trade-off

Reach for a CSV when…

- Data is read-mostly
- Single-user access
- Fits comfortably in memory
- Inter-system exchange
- ML datasets (Kaggle, Hugging Face)

Stop and use a real DB when…

- Two writers at once (no locking!)
- Indexed lookups at scale
- Transactions (all-or-nothing)
- Referential integrity
- Anything approaching a GB live

Flat Files in Back-End Development

Three places you will meet them

1. Configuration — .env, .yaml, .json read at start-up
2. Data interchange — nightly CSV dumps to other systems
3. Log files — every HTTP request produces one line

CSV Parsing — The Gotchas

Never split on commas by hand

- ⚠ Names like "Smith, Jr." contain commas
- ⚠ Newlines can appear INSIDE quoted fields
- ⚠ Non-ASCII characters → encoding errors on Windows
- ⚠ CRLF vs LF → silent data drops
- ⚠ ALWAYS use your language's standard CSV parser

KEY

Flat files are simple, portable, and dangerous.

Perfect for config, exports, and interchange. Disaster for concurrent writes or indexed queries.

Always use your language's standard CSV parser — never split on commas by hand.

LESSON 3

SQL

Structured Query Language · the fifty-year-old king

Why SQL Still Rules

If you only remember one back-end technology…

- Over 50 years old, still the industry default
- Every bank, airline, tax agency speaks SQL
- "Database" in industry usually means SQL database
- ANSI / ISO standard since 1986
- It will outlive your programming career

The Beautiful Idea of SQL

Declarative, not imperative

```
SELECT full_name FROM Student
WHERE country = 'Spain'
ORDER BY enrolled_on DESC;
```

You tell the engine WHAT you want, not HOW to get it. The engine — debugged by thousands of engineers over five decades — picks the best algorithm and index for you. That is declarative programming.

Five Sub-Languages of SQL (1/2)

DDL, DML, DQL — the bread and butter

1. DDL — Data Definition Language · CREATE, ALTER, DROP
2. DML — Data Manipulation Language · INSERT, UPDATE, DELETE
3. DQL — Data Query Language · SELECT (90% of daily work)

Five Sub-Languages of SQL (2/2)

DCL and TCL — the safety layer · bank transfers are TCL in action

1. DCL — Data Control Language · GRANT, REVOKE
2. TCL — Transaction Control Language · BEGIN, COMMIT, ROLLBACK

ACID · The Four-Letter Contract

When a bank loses money in the news, one of these was violated

- ★ **Atomicity — all or nothing**
- ★ **Consistency — always a valid state**
- ★ **Isolation — concurrent transactions do not collide**
- ★ **Durability — once committed, survives power loss**

Keys and Relationships

Tables connect by data, not pointers

- Primary Key (PK) — uniquely identifies a row
- Foreign Key (FK) — PK of another table, used as a reference
- Composite PK — two columns together form the unique identifier
- Engine enforces: no orphaned references, no duplicates
- Student has id · each Enrolment carries that id as student_id

Major SQL Engines

Pick one, get productive

Engine	Licence	Niche
PostgreSQL	Open source	General-purpose, full-featured
MySQL / MariaDB	Open source	Web applications, LAMP stack
SQLite	Public domain	Mobile, embedded, local
Oracle Database	Commercial	Enterprise, banking
SQL Server	Commercial	Microsoft .NET shops
IBM Db2	Commercial	Mainframe, hybrid cloud

KEY

Default to SQL. Always.

Write declarative queries, use ACID transactions, connect tables through primary and foreign keys.

If you only remember one back-end technology from this course, make it this one.

LESSON 4

NoSQL

Not Only SQL · scale, schema, speed

Why NoSQL Emerged

Born from a very real engineering problem

- The 2005 scale wall at Google, Amazon, Facebook
- Relational engines scale vertically — bigger box, same engine
- Global-scale apps need horizontal scale — many machines
- Trade-off: give up SOME guarantees, gain MASSIVE scale
- NoSQL = "Not Only SQL", a companion — not a replacement

Family 1 · Document Stores

Self-describing JSON-like records

- Each document has its own shape — schema-flexible
- Natural fit for hierarchical data (order → items → options)
- Examples: MongoDB, Couchbase, Firestore
- Great for: CMS, product catalogues, mobile back-ends

Family 2 · Key-Value Stores

The simplest possible database

- Give a key, get a value — that is it
- All operations are O(1)
- Typically in-memory, microsecond reads
- Examples: Redis, Memcached, DynamoDB core mode
- Great for: caching, sessions, rate limiting, counters

Family 3 · Wide-Column Stores

Sparse, wide, distributed

- Millions of sparse columns per row — empty cells are free
- Built to scale horizontally across datacenters
- Write-heavy workloads, time-series data
- Examples: Apache Cassandra, HBase, Google Bigtable
- Great for: IoT ingest, logs, massive time-series

Family 4 · Graph Databases

Nodes and edges as first-class citizens

- Relationships are stored, not computed by joins
- Multi-hop traversals are fast
- Query languages: Cypher, Gremlin, SPARQL
- Examples: Neo4j, Amazon Neptune, ArangoDB
- Great for: social networks, fraud detection, recommendations

CAP Theorem · Brewer, 2000

In a distributed system you get two of three

- **Consistency — every read sees the latest write**
- **Availability — every request gets a response**
- **Partition tolerance — works when the network splits**
- Partitions are reality — real choice is C vs A
- Banks → CP (reject rather than risk)
- Social networks → AP (stay up, sync later)

BASE · The Counterweight to ACID

Amazon lets the stock counter disagree for a few ms — they chose availability

- **Basically Available — the system stays up**
- **Soft state — the state can change without input**
- **Eventually consistent — it gets there, just not instantly**

When to Reach for NoSQL

The honest industry rule

Pick the family by shape of data

- Document → CMS, catalogue, mobile back-end
- Key-Value → cache, session, counter
- Wide-column → IoT, logs, time-series
- Graph → relationships first

When to NOT reach for NoSQL

- Data is tabular and relational
- You have fewer than a million rows
- You need ACID transactions
- Default to SQL, upgrade only with a reason

KEY

NoSQL is Not Only SQL — a specialised tool, not a replacement.

Four families (document, key-value, wide-column, graph), each a different trade-off on the CAP triangle. Go with BASE consistency when scale matters more than strict ACID.

LESSON 5

Hybrid & Big Data

Polyglot persistence · MapReduce · data lakes

Polyglot Persistence

Example · building Spotify with the right tool per workload

- SQL → users, subscriptions, payments
- Redis → sessions, play counters
- MongoDB / Elasticsearch → music catalogue, search
- Graph / Vector → recommendations
- S3 data lake → raw event logs

Multi-Model Databases

One engine, multiple paradigms · small teams love this

- PostgreSQL — relational + JSON + full-text + vector
- MongoDB — document + graph lookups + time-series
- Oracle — relational + JSON + graph + spatial
- Azure Cosmos DB — SQL + document + key-value + graph APIs

Big Data · The 5 V's

CERN's LHC produces ≈ 1 petabyte per second when running

1. Volume — terabytes → petabytes → exabytes
2. Velocity — millions of events per second, continuously
3. Variety — text, JSON, images, audio, graph, geo
4. Veracity — can the data be trusted?
5. Value — can we turn it into money, science, decisions?

MapReduce · In One Slide

Two functions, thousands of machines

```
INPUT chunks  →  [MAP]  →  (key, value)  →  shuffle  →  [REDUCE]  →  OUTPUT
```

MAP runs on each chunk and emits (key, value) pairs. REDUCE groups by key and combines values. Invented at Google (2004), open-sourced as Apache Hadoop, succeeded by Apache Spark — same pattern, in-memory, much faster.

Big Data Platforms You Will Meet

Ingestion → storage → compute → analytics

- Ingestion — Apache Kafka (distributed event log)
- Storage — AWS S3, Google Cloud Storage, Azure Data Lake, HDFS
- Compute — Apache Spark, Hadoop MapReduce, Flink
- Warehouses — Snowflake, BigQuery, Amazon Redshift
- BI / dashboards — Looker, Power BI, Tableau

Data Warehouse vs Data Lake

Two landing zones for big data · Lakehouse = recent hybrid

Property	Data Warehouse	Data Lake
Data shape	Structured, cleaned	Raw, messy, any format
Query language	SQL	Anything (Spark, Python, SQL)
Cost per TB	High	Low
Main users	Business analysts	Data scientists, ML teams
Examples	Snowflake, BigQuery	S3, Azure Data Lake, GCS

KEY

Modern back-ends are hybrid by default.

Pick the right database per workload (polyglot persistence). For data beyond a single machine, lean on the 5 V's framework and the MapReduce pattern — the same model from a laptop to a thousand-node cluster.

Session Wrap-Up · Matching Tool to Problem

Your job is not to memorise names — it is to develop judgement

- → Read-mostly, fits in memory, interchange → flat file
- → Structured, transactional, integrity required → SQL
- → Schema-flexible, high-volume, distributed → NoSQL
- → Many of the above → polyglot persistence
- → Beyond a single machine's reach → Big Data pipelines

Further Reading · Official Sources

All links in the accompanying lecture document

Standards & engines

- ANSI · ISO · Apache Software Foundation
- PostgreSQL · MySQL · SQLite
- Oracle · SQL Server · IBM Db2
- MongoDB · Redis · Cassandra · Neo4j

Cloud & Big Data

- AWS · Google Cloud · Microsoft Azure
- Snowflake · Databricks
- Apache Hadoop · Spark · Kafka
- Kaggle · Hugging Face

Disclaimer · Cyber.SoHo Educational Hub

The fine print that protects us both

- Intellectual property — content owned by Cyber.SoHo
- No affiliation with any third-party vendor or platform
- Fair use — third-party names cited for education only
- Link safety — verify all external URLs with VirusTotal
- Liability — code is for learning, not production
- Ethics — use what you learn responsibly, respect GDPR

Thank you.

Questions?

From the Industry to the Classroom — Building the IT's Future, Today and Together!